



ÉCOLE NATIONALE SUPÉRIEURE
D'INFORMATIQUE ET D'ANALYSE DES SYSTÈMES
- RABAT

DEPARTEMENT D'INGÉNIERIE DE SYSTEME EMBARQUÉ
PROJET DE FIN D'ANNÉE

Systeme intelligent de surveillance d'un
enfant

Élèves :

Hamidou GUINDO
Roger CAMARA

Encadrant :

Faissal ELBOUANANI

Jury :

Z.E.A Alaou ISMAILI
A.KRIOULE

Année Universitaire 2025-2026

Remerciements

Nous exprimons nos sincères remerciements à notre professeur encadrant, Monsieur Faissal El Bouanani, pour son accompagnement, sa disponibilité et ses précieux conseils tout au long de la réalisation de ce projet de fin d'année. Son regard critique et ses orientations nous ont permis d'améliorer la qualité de notre travail et d'enrichir notre réflexion. De suite, nous aimerions remercier les membres du Jury qui ont accepté évaluer notre projet, leurs retour élogieux et leurs encouragements nous pousse à vouloir mieux faire. Enfin, nos remerciements vont également au corps professoral de notre département ainsi qu'au corps administratif de l'Ecole Nationale Supérieure de l'Informatique et de l'Analyse des Systèmes -ENSIAS pour l'atmosphère bienveillante dans laquelle ils nous ont permis de travailler.

Résumé

Ce projet de fin d'année consiste à développer un système intelligent de surveillance d'un enfant basé sur le développement web, l'IoT et l'intelligence artificielle. L'objectif est d'aider les parents à surveiller leur bébé en temps réel et d'être alertés en cas de comportements inhabituels. Le système repose sur une raspberry pi qui centralise les données collectées par des capteurs (son, mouvement, température) et une caméra intégrée. Les capteurs détectent les bruits alentour, l'absence de mouvement et une température anormale, tandis que la caméra capture et analyse les images tout en permettant un streaming vidéo en direct. Une logique IA est appliquée pour analyser les données et reconnaître automatiquement les comportements anormaux. Les résultats sont affichés sur un dashboard web développé avec Flask et React, offrant une visualisation en temps réel, des graphiques d'historique et l'état du bébé (normal, vigilance, alerte). En cas d'anomalie, le système déclenche des actions : notifications automatiques (email, mobile), alarme sonore et affichage sur le dashboard. La sécurité repose sur la fiabilité de la détection IA, la confidentialité des données et la réactivité des alertes. Le projet a été mené selon une méthodologie agile scrum sur une durée de 7 semaines (04 mai – 22 juin), avec des phases allant de la conception à l'intégration des capteurs, le développement du dashboard, la mise en place des notifications et les tests de validation. Ce système constitue une solution IoT innovante, combinant capteurs, vision par caméra, analyse intelligente par IA et interface web pour assurer une surveillance sécurisée et proactive de l'enfant.

Abstract

This project presents the design and implementation of an intelligent baby monitoring system called "Œil des Parents". The system integrates multiple sensors (camera, motion, sound, and temperature) connected to a Raspberry Pi, which serves as the central unit for data acquisition and preprocessing. The collected information is transmitted to a modular Python-based backend that coordinates audio-visual analysis and artificial intelligence models to evaluate the baby's condition and compute a danger score. A React.js frontend provides parents with a real-time dashboard displaying the baby's status, alerts, predictions, and system health indicators. The proposed solution ensures continuous supervision, reduces false alarms through data fusion, and offers a user-friendly interface for parental decision-making. This work demonstrates the feasibility of combining embedded systems, AI, and web technologies to enhance child safety and parental peace of mind.

Introduction générale

La sécurité et le bien-être des enfants représentent une préoccupation majeure pour les parents, en particulier durant leur sommeil. Avec l'évolution des technologies IoT et des systèmes embarqués, il est désormais possible de concevoir des solutions intelligentes capables de surveiller en temps réel et de détecter automatiquement des comportements inhabituels. C'est dans ce contexte que s'inscrit notre projet de fin d'année. Notre projet consiste à développer un système intelligent de surveillance d'un enfant, basé sur des capteurs IoT, une caméra Raspberry Pi et des algorithmes d'intelligence artificielle. Ce système vise à collecter et analyser des données (son, mouvement, température) afin de détecter des anomalies telles que les pleurs ou l'absence de mouvement, et à alerter les parents par différents moyens (emails, notifications mobiles, alarmes sonores). La problématique que nous abordons est la suivante : comment concevoir un système fiable, en temps réel, capable de surveiller un enfant et de prévenir les parents en cas de comportement anormal, tout en garantissant la confidentialité et la sécurité des données ? Notre hypothèse est qu'une combinaison de capteurs IoT, de caméra et d'outils d'IA intégrés dans une architecture distribuée peut fournir une solution efficace, proactive et sécurisée pour la surveillance d'un enfant appelé "Oeil des parents".

Table des matières

Remerciements	2
Résumé	3
Abstract	4
Introduction générale	5
1 Contexe du travail	10
Chapitre 1 : Contexe du travail	10
1.1 Introduction	10
1.2 Contexe et definition du probleme	10
1.3 Solution de problème	10
1.4 Etudes des besoins	10
1.5 Listes des acteurs	10
1.6 Les besoins fonctionnels	11
1.7 Les besoins non fonctionnels	11
1.8 Méthodologie	11
1.8.1 Modèle de cycle de vie en Agile	11
1.9 Justification du choix	11
1.10 Conclusion	12
2 Analyse et Conception	13
Chapitre 2 : Analyse et Conception	13
2.1 Introduction	13
2.2 Analyse	13
2.2.1 Ouverture	13
2.2.2 UML	13
2.2.3 Diagramme de classe	13
2.2.4 Diagramme de cas d'utilisation	14
2.2.5 Diagramme d'état transition	15
2.2.6 Diagramme d'activité	16
2.2.7 Diagramme de sequence	18
2.3 Conclusion	18
3 Réalisation	19
Chapitre 3 : La réalisation	19

3.1	Introduction	19
3.2	Composants utilisés	19
3.2.1	Architecture du système	20
3.2.2	Montage du dispositif	21
3.3	Raspberry Pi	22
3.3.1	Installation du système d'exploitation	22
3.3.2	Configuration de la Raspberry Pi	22
3.3.3	Installation des bibliothèques	23
3.3.4	Acquisition des données	23
3.3.5	IMAGE de l'exécution du programme main.py de la Raspberry Pi dans le terminal	23
3.3.6	Transmission des données vers le backend	23
3.4	Backend	24
3.4.1	Organisation du backend	24
3.4.2	Ai engine	24
3.4.3	Pc audio client	25
3.4.4	App	25
3.4.5	Choix de la technologie	26
3.4.5.1	Visual Studio Code	26
3.4.5.2	Python	27
3.5	Frontend	27
3.5.1	INTERFACE DU DASHBOARD	29
3.5.2	ETAT DU BEBE	30
3.5.3	AUTRES PAGES DISPONIBLES DANS LA BARRE LATÉRALE	32
	Conclusion Générale	33
	Conclusion Générale	33
	Bibliographie	34
	Annexe	35

Table des figures

1.1	Diagramme UML du système	11
2.1	Diagramme de classe	14
2.2	Diagramme de cas d'utilisation	15
2.3	Diagramme de transition	16
2.4	Diagramme d'activité	17
2.5	Diagramme de sequence	18
3.1	Architecture du systeme	21
3.2	Montage des materiels	22
3.3	Image d'execution	23
3.4	La structure du backend	24
3.5	image d'execution de <i>pc_audio_client</i>	25
3.6	image d'execution de app.py	26
3.7	visual studio code	26
3.8	Python	27
3.9	La structure du frontend	28
3.10	dasbord	29
3.11	dasbord	30
3.12	Tableaux de bord 3, 4 et 5	31

Liste des tableaux

3.1	Composants matériels utilisés pour la réalisation du système	20
3.2	Organisation des fichiers du backend	24
3.3	Organisation des fichiers du frontend	28

Chapitre 1

Contexe du travail

1.1 Introduction

La surveillance des enfants, notamment durant leur sommeil, est une préoccupation essentielle pour les parents. Les méthodes traditionnelles reposent souvent sur une vigilance humaine constante, ce qui peut être contraignant et peu fiable. Avec l'évolution des technologies IoT et des systèmes embarqués, il est désormais possible de concevoir des solutions intelligentes capables de collecter, analyser et interpréter des données en temps réel pour assister les parents.

1.2 Contexe et definition du probleme

Le problème posé est celui de la détection automatique des comportements inhabituels chez un enfant pendant son sommeil. Les pleurs, l'absence de mouvement ou une température anormale sont des signaux critiques qui doivent être identifiés rapidement. Les solutions existantes manquent souvent de précision ou d'intégration. Notre projet vise à combler ce manque en proposant un système complet et intelligent.

1.3 Solution de problème

Nous proposons un système IoT intelligent appelé oeil des parents, basé sur une Raspberry Pi, intégrant des capteurs (son, mouvement, température) et une caméra. Les données collectées sont analysées par une logique d'intelligent artificielle afin de détecter les comportements anormaux. Les résultats sont affichés sur un dashboard web en temps réel et des alertes automatiques (emails, notifications mobiles, alarmes sonores) sont envoyées aux parents.

1.4 Etudes des besoins

Le projet répond à la fois au besoin des parents, qui souhaitent une surveillance fiable et continue de leur enfant, et à celui de l'administrateur technique, chargé de configurer et maintenir le système. L'objectif est d'assurer une surveillance optimale en temps réel, tout en minimisant les risques liés à une mauvaise détection ou à un manque de réactivité.

1.5 Listes des acteurs

Notre système implique différents acteurs qui interagissent avec la plateforme : parents(utilisateurs finaux) : consultent le dashboard, reçoivent les notifications et visualisent les données.

lisent le flux vidéo. Administrateurs : configure les capteurs, gère le système et supervise le bon fonctionnement.

1.6 Les besoins fonctionnels

Les besoins fonctionnels du système de surveillance incluent : Collecte des données via capteurs IoT (son, mouvement, température). Capture et traitement d'images via caméra Raspberry Pi. Affichage en temps réel des données et de l'état de l'enfant sur un dashboard web. Notifications automatiques (email, mobile, alarme sonore) en cas d'anomalie. Stockage des données et génération d'historiques consultables. Détection intelligente des comportements inhabituels grâce à l'IA.

1.7 Les besoins non fonctionnels

Les besoins non fonctionnels incluent : Rapidité de traitement : les alertes doivent être générées en temps réel. Performance : le système doit gérer efficacement plusieurs flux de données simultanés. Convivialité : interfaces simples, ergonomiques et adaptées aux parents. Sécurité : accès sécurisé aux données, distinction claire entre droits de l'administrateur et des utilisateurs. Fiabilité : le système doit signaler les erreurs et garantir la continuité du service.

1.8 Méthodologie

1.8.1 Modèle de cycle de vie en Agile

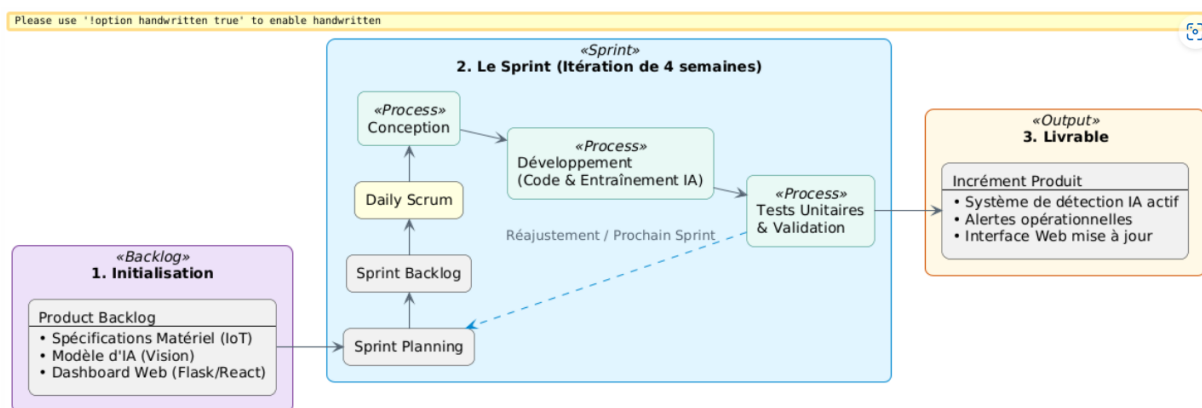


FIGURE 1.1 – Diagramme UML du système

1.9 Justification du choix

Le choix de la méthodologie agile scrum pour le développement de notre système de surveillance s'explique par la nature évolutive et multidisciplinaire du projet. En effet,

celui-ci combine plusieurs volets : intégration de capteurs IoT, traitement d'images par caméra, analyse intelligente via IA, développement d'un dashboard web et mise en place de notifications en temps réel. La méthode Agile Scrum permet une organisation en sprint courts et itératifs, favorisant l'adaptation rapide aux imprévus et l'amélioration continue des fonctionnalités. Chaque sprint intègre la conception, le développement, les tests et la validation, ce qui garantit une progression régulière et une traçabilité des résultats. Ce choix est justifié par : La flexibilité : possibilité d'ajuster les priorités selon les besoins des utilisateurs (parents et administrateur). La validation continue : les fonctionnalités critiques (détection IA, alertes, dashboard) sont testées dès leur implémentation. La qualité du produit final : grâce à l'intégration des tests et retours à chaque étape. La collaboration : le travail en binôme est facilité par une répartition claire des tâches et une communication régulière. Ainsi, la méthodologie Agile Scrum assure une gestion efficace du projet, une meilleure réactivité face aux contraintes techniques et une garantie de livrer un système fiable et sécurisé dans le délai imparti.

1.10 Conclusion

Ce chapitre a posé les bases du projet en présentant le contexte, la problématique, les besoins, la méthodologie et la planification. Il constitue le point de départ de la conception du système intelligent de surveillance d'un enfant, combinant IoT, intelligence artificielle et interface web pour une supervision sécurisée et proactive.

Au-delà de la simple description des éléments initiaux, cette étape a permis de clarifier les objectifs du projet, de définir les contraintes techniques et organisationnelles, ainsi que d'identifier les solutions les plus adaptées pour répondre aux besoins des parents. Elle a également servi à établir une méthodologie rigoureuse, garantissant que chaque phase du développement soit structurée et orientée vers l'atteinte des résultats attendus.

Ainsi, ce chapitre ne se limite pas à une introduction théorique, mais constitue une véritable fondation sur laquelle repose la suite du travail. En reliant les concepts d'IoT, d'IA et de développement web, il ouvre la voie à une réalisation concrète et démontre que la problématique initiale peut être résolue par une approche technologique moderne et intégrée.

Chapitre 2

Analyse et Conception

2.1 Introduction

Dans ce chapitre, nous identifions les fonctionnalités de notre système pour chaque type d'utilisateur. Cette étape consiste à recenser les besoins fonctionnels et non fonctionnels, puis à les traduire en cas d'utilisation. L'analyse des besoins permet de clarifier les attentes des parents (utilisateurs finaux) et de l'administrateur technique, afin de guider la conception et le développement du système.

2.2 Analyse

2.2.1 Ouverture

Cette partie est consacrée aux étapes fondamentales pour le développement de notre système de surveillance intelligente. Pour la conception et la réalisation de notre application, nous avons choisi de modéliser en nous appuyant sur le formalisme UML, qui offre une flexibilité marquante et s'exprime par l'utilisation de différents diagrammes. Concepts Fondamentaux

2.2.2 UML

Dans notre projet de surveillance intelligente, l'UML a été utilisé comme outil de modélisation pour représenter clairement les besoins et les interactions du système. Concrètement, l'UML nous a permis de : Définir les cas d'utilisation entre les acteurs (parents et administrateurs) et le système. Décrire la structure du système à travers un diagramme de classes (entités : enfant, capteur, caméra, notification, utilisateur). Illustrer les flux d'événements et les échanges entre composants via des diagrammes de séquence. Représenter les processus internes grâce aux diagrammes d'activité. Ainsi, l'UML n'est pas seulement une méthode théorique, mais un outil pratique qui nous a guidés dans la conception et la réalisation du système, en assurant une meilleure communication et une traçabilité des choix techniques.

2.2.3 Diagramme de classe

Ce diagramme illustre l'architecture logicielle de notre système. Il met en évidence quatre blocs principaux : la gestion des utilisateurs, le matériel IoT (capteurs et caméra), le cœur d'analyse décisionnelle avec le modèle IA de vision, et la gestion des notifications. Les attributs, méthodes et multiplicités assurent une organisation claire et une traçabilité fiable des données liées à l'enfant.

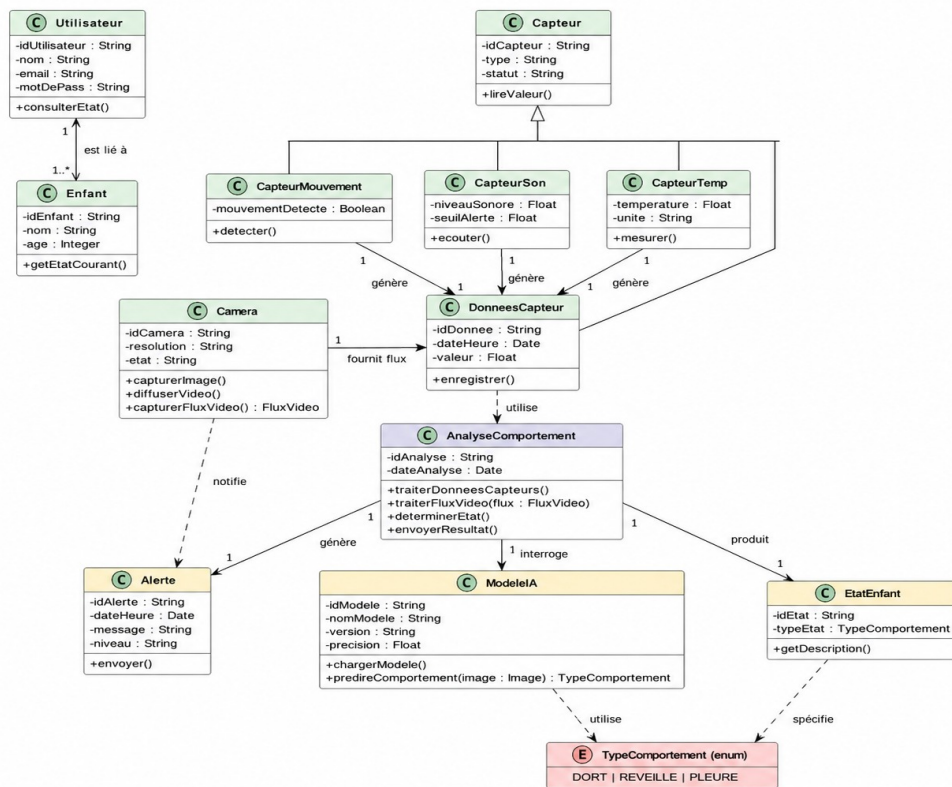


FIGURE 2.1 – Diagramme de classe

2.2.4 Diagramme de cas d'utilisation

Ce diagramme illustre de façon détaillée les principales fonctionnalités offertes par le système du point de vue des utilisateurs. Il met en évidence l'acteur central, le parent, qui constitue le bénéficiaire direct de l'application, ainsi que les acteurs techniques tels que l'intelligence artificielle et les capteurs, organisés autour du tableau de bord principal. La représentation graphique souligne non seulement la place du parent dans l'écosystème, mais aussi la manière dont il interagit avec les différents modules de l'application.

À travers les relations d'extension, le schéma montre de manière synthétique et structurée les multiples possibilités offertes : consultation de la vidéo en temps réel, réception et gestion des alertes, accès à l'historique des événements, ou encore activation du mode sombre pour améliorer l'expérience utilisateur. Ces interactions traduisent la richesse fonctionnelle du système et la diversité des scénarios d'utilisation.

Enfin, le diagramme met en lumière le rôle des alertes intelligentes transmises par l'IA, qui analyse les données issues des capteurs pour générer des notifications pertinentes. Cette articulation entre acteurs humains et techniques illustre la complémentarité entre surveillance automatisée et contrôle parental, garantissant une supervision efficace et une meilleure réactivité face aux situations critiques.

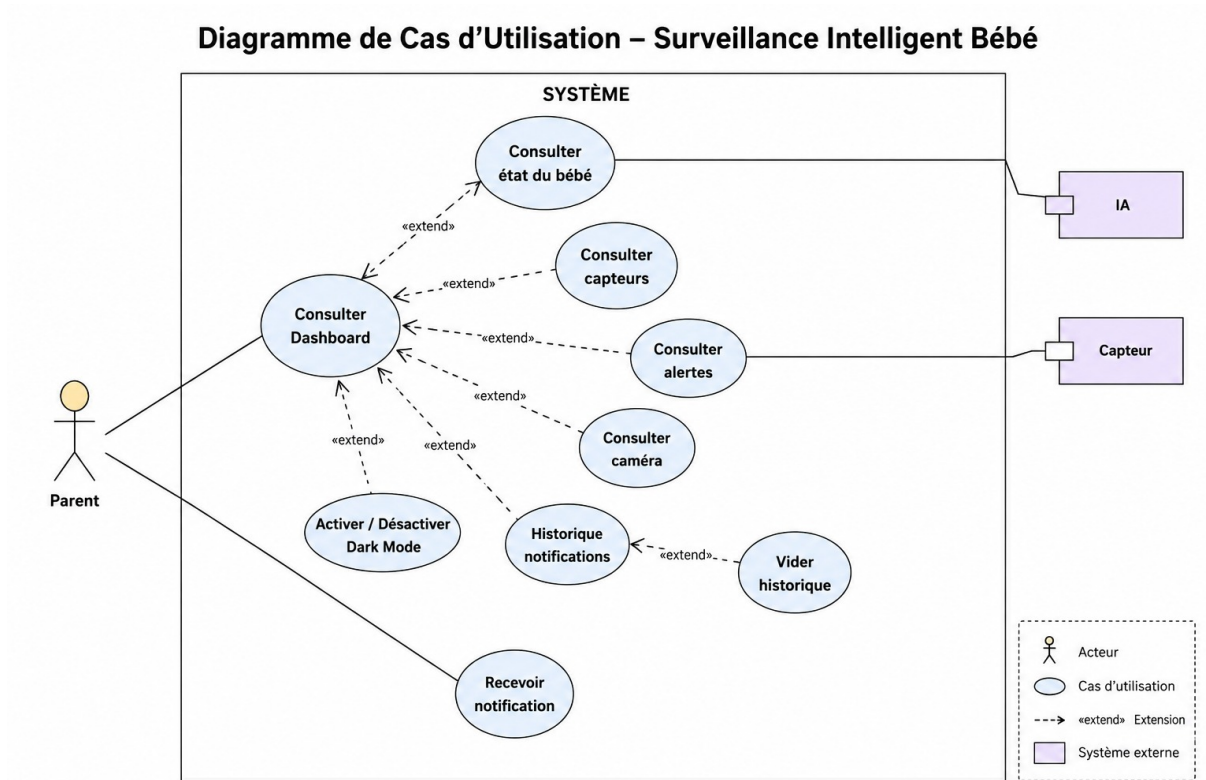


FIGURE 2.2 – Diagramme de cas d'utilisation

2.2.5 Diagramme d'état transition

Ce diagramme d'états modélise le comportement dynamique du système en représentant la situation de l'enfant à chaque instant. Il montre le passage automatique de l'état initial Réveillé vers les états Dort ou Pleure, en fonction des signaux captés par les capteurs. Les transitions reposent sur des conditions de garde strictes, calculées en temps réel, telles qu'un niveau sonore élevé, des mouvements fréquents ou au contraire une absence prolongée d'activité. Ces conditions permettent de déclencher les alertes appropriées et d'assurer une surveillance continue.

Au-delà de la simple représentation graphique, ce diagramme met en évidence la logique de décision intégrée dans le système : chaque état traduit une situation concrète de l'enfant, et chaque transition correspond à une réaction immédiate du dispositif. Ainsi, le passage de Réveillé à Pleure illustre la détection d'un comportement anormal, tandis que la transition vers Dort traduit une situation normale et apaisée.

En synthèse, ce modèle visuel ne se limite pas à décrire des états statiques, mais il reflète la capacité du système à analyser en permanence les données issues des capteurs et à adapter son comportement en conséquence. Il constitue donc un outil essentiel pour comprendre la logique interne de l'application et la manière dont elle garantit une surveillance intelligente et proactive.

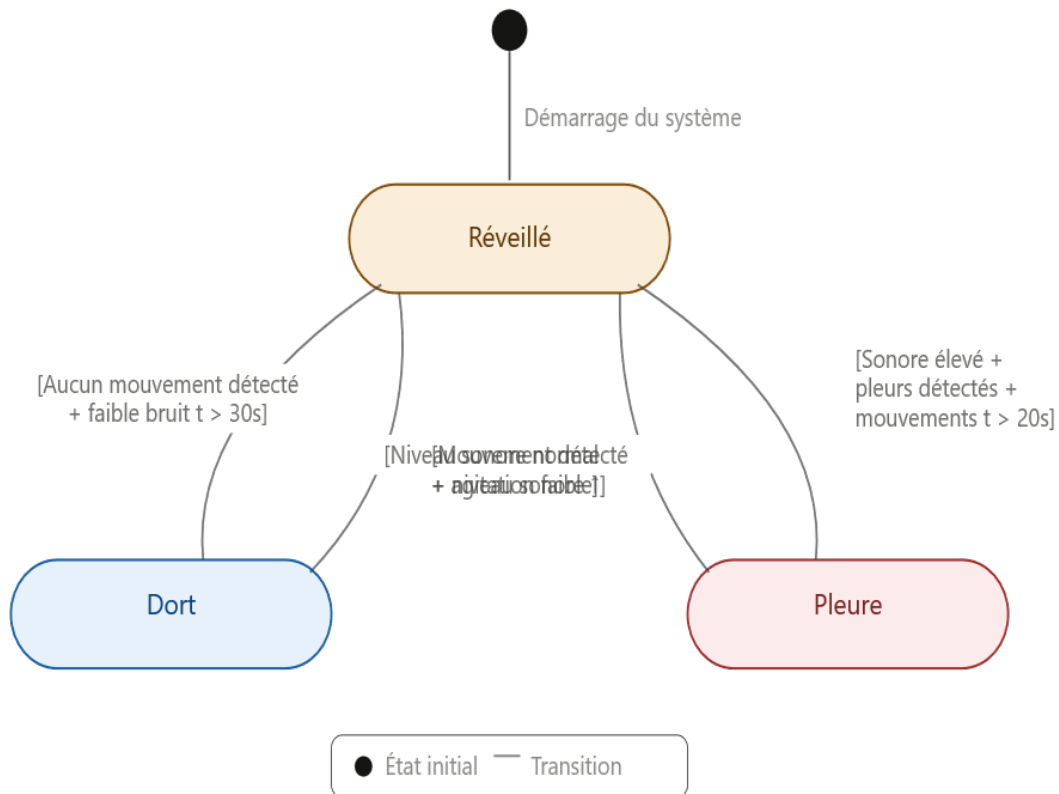


FIGURE 2.3 – Diagramme de transition

2.2.6 Diagramme d'activité

Ce diagramme de séquence illustre de manière approfondie le flux d'exécution du système de surveillance, en mettant en évidence les enchaînements séquentiels et parallèles des traitements. Il modélise le comportement interne de l'application depuis le démarrage du dispositif, en passant par la collecte des données issues des capteurs, leur analyse par le moteur d'intelligence artificielle, jusqu'à la mise à jour des interfaces d'affichage destinées au parent.

La représentation graphique permet de visualiser clairement la chronologie des interactions entre les différents composants : les capteurs transmettent en continu les informations, le module d'analyse décisionnelle applique les règles de surveillance, puis les résultats sont relayés vers le tableau de bord. Les transitions parallèles montrent la capacité du système à gérer simultanément plusieurs flux, tels que la détection sonore et la reconnaissance visuelle, garantissant une réactivité optimale.

Ce diagramme met également en lumière la robustesse du système, qui ne se limite pas à un simple enchaînement linéaire, mais intègre des mécanismes de synchronisation et de mise à jour en temps réel. Ainsi, chaque événement critique déclenche une cascade de traitements coordonnés, assurant que les alertes et l'état de l'enfant soient reflétés instantanément dans l'interface utilisateur.

Diagramme d'Activité – Surveillance Intelligent Bébé

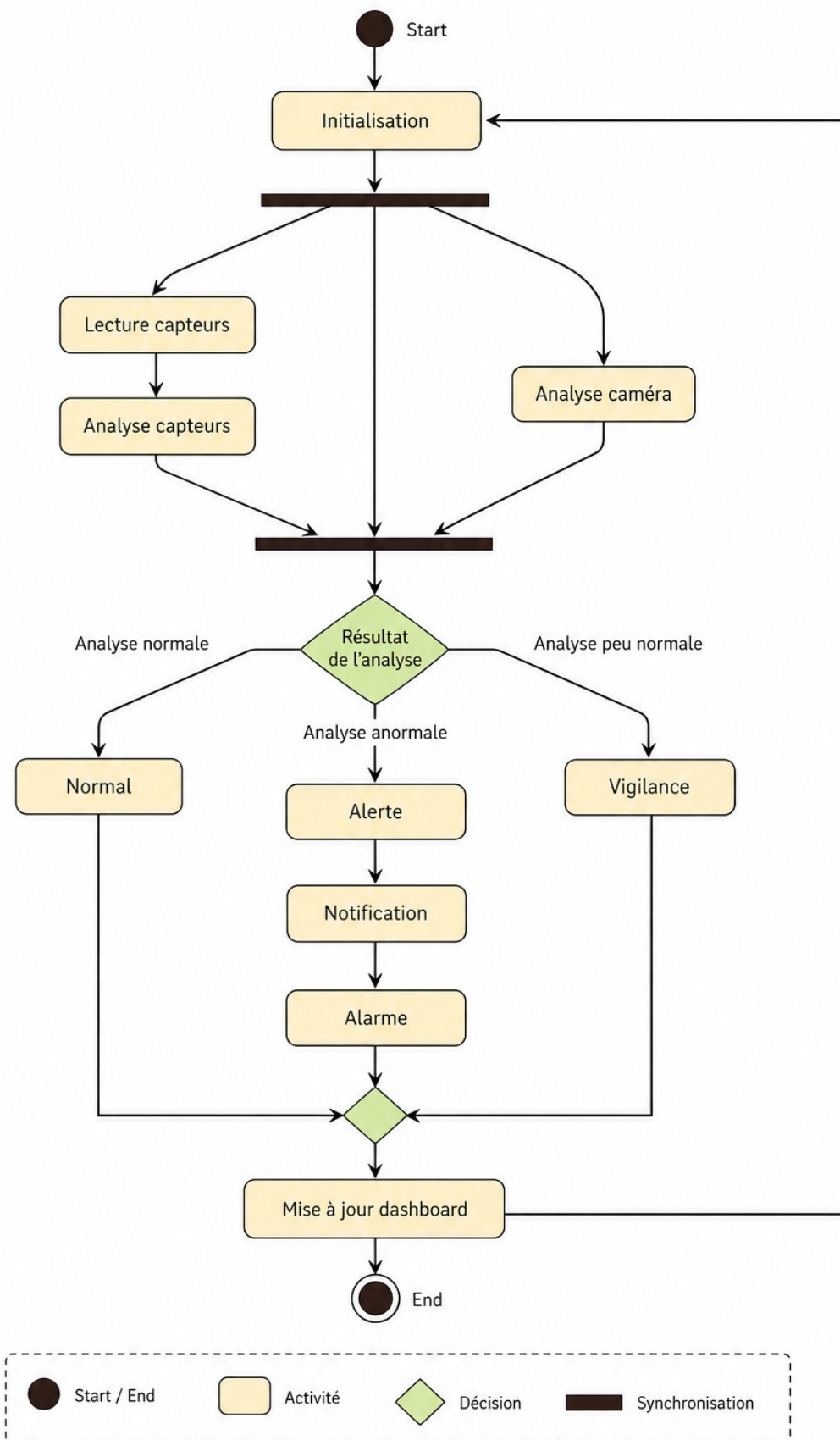


FIGURE 2.4 – Diagramme d'activité

2.2.7 Diagramme de sequence

Le diagramme de séquence ci-dessous illustre la dynamique générale du système et la chronologie des interactions entre les composants matériels, les scripts de traitement et l'interface utilisateur lors d'un événement critique. Il met en évidence la succession ordonnée des messages échangés, ainsi que les parallélismes qui assurent une exécution fluide et rapide.

On y observe notamment la manière dont les capteurs transmettent les données, comment les scripts de traitement appliquent les règles de surveillance, et enfin comment l'interface utilisateur est mise à jour pour informer le parent en temps réel. Cette représentation permet de comprendre la logique interne du dispositif, en montrant que chaque étape est synchronisée et contribue à la réactivité globale du système.

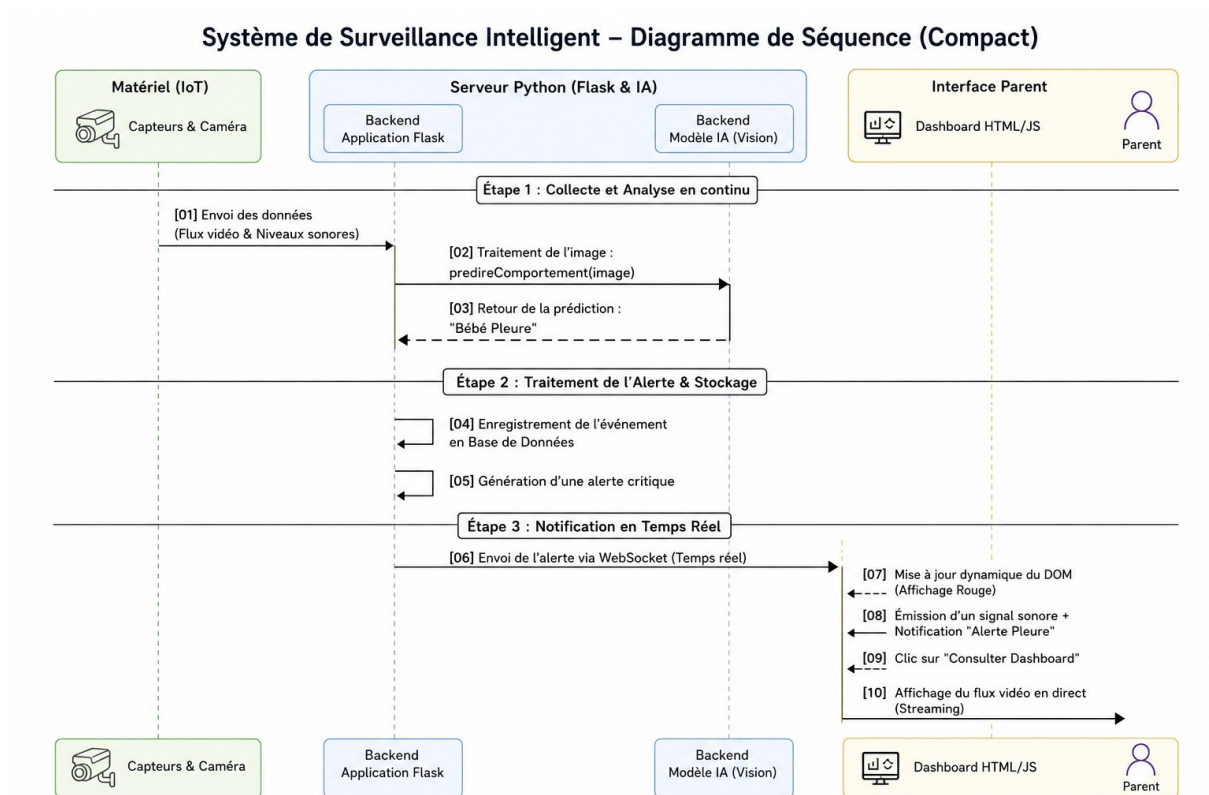


FIGURE 2.5 – Diagramme de sequence

2.3 Conclusion

La phase conceptuelle constitue une étape essentielle dans la réalisation de notre projet de surveillance intelligente. Elle permet de clarifier les besoins, de définir la structure du système et de préparer l'implémentation de la base de données. Cette analyse et conception facilitent la transition vers la phase de développement, où nous présenterons les outils et moyens techniques utilisés pour mettre en œuvre l'application.

Chapitre 3

Réalisation

3.1 Introduction

Après avoir achevé l'étape de conception de notre système de surveillance oeil des parents, nous entamons dans ce chapitre la phase de réalisation. Celle-ci a pour objectif de présenter en détail le travail effectué et de montrer comment les choix conceptuels ont été traduits en solutions concrètes et opérationnelles.

Nous commencerons par préciser l'environnement matériel et logiciel utilisé, en décrivant les composants physiques (Raspberry Pi, capteurs, caméra, périphériques) ainsi que les outils de développement (Python, bibliothèques spécialisées, frameworks web). Cette étape est essentielle car elle constitue la base technique sur laquelle repose l'ensemble du système.

Ensuite, nous détaillerons les différentes phases de développement et d'intégration :

Montage matériel : connexion et configuration des capteurs, caméra et périphériques.

Développement logiciel : mise en place du backend en Python, intégration du moteur d'intelligence artificielle, gestion des flux audio et vidéo.

Frontend et interface utilisateur : création du tableau de bord en React.js, organisation des pages et modules, communication en temps réel avec le backend.

Tests et validation : simulation des scénarios critiques, vérification de la fiabilité des alertes et de la cohérence des données affichées.

Enfin, nous mettrons en évidence la manière dont l'interface utilisateur permet aux parents de consulter en temps réel l'état du bébé, de recevoir des alertes intelligentes et de bénéficier d'une supervision continue. Cette phase de réalisation constitue donc une étape clé, car elle démontre la faisabilité technique du projet et traduit les modèles théoriques en un prototype fonctionnel, prêt à être évalué et amélioré.

3.2 Composants utilisés

L'ensemble de ces composants constitue l'ossature matérielle du système «Eil des parents». Chaque élément joue un rôle spécifique et indispensable, depuis la collecte des données brutes (images, sons, mouvements, température) jusqu'à leur traitement et leur transmission vers le backend. La combinaison de ces dispositifs assure une surveillance fiable et en temps réel, tout en garantissant la sécurité et le confort du bébé.

Composants	Rôles
Raspberry Pi 3 B	Unité centrale du système : collecte les données des capteurs, exécute les algorithmes d'analyse (vision et audio), pilote les périphériques et transmet les informations vers le backend.
Caméra IMX477	Capture en temps réel les images du bébé afin de détecter la présence, les mouvements, l'ouverture ou la fermeture des yeux ainsi que l'état général du bébé.
Capteur de mouvement PIR HC-SR501	Détecte les mouvements corporels dans la zone de surveillance afin de confirmer l'activité ou l'absence d'un intrus dans la chambre du bébé.
Capteur de son KY-027	Mesure les variations du niveau sonore autour du bébé (bruit/silence).
Capteur de température KY-028	Mesure la température de l'environnement afin de vérifier que les conditions restent confortables pour le bébé.
LED (verte, jaune, rouge)	Indiquent l'état du système (verte : normal, jaune : vigilance, rouge : critique).
Résistances 220 Ω	Protègent les LED contre les surintensités en limitant le courant électrique fourni par la Raspberry Pi.
Breadboard	Permet de réaliser les connexions électriques entre les différents composants sans soudure.
Carte microSD 16 Go	Stocke le système d'exploitation Raspberry Pi OS, les programmes Python, les modèles d'intelligence artificielle ainsi que les fichiers nécessaires au fonctionnement du système.
Ordinateur portable	Sert au développement, à la configuration de la Raspberry Pi, au déploiement du projet et à la consultation de l'interface web de supervision.

TABLE 3.1 – Composants matériels utilisés pour la réalisation du système

3.2.1 Architecture du système

Ce schéma d'architecture illustre l'organisation globale du projet «Eil des parents» et la manière dont les différents modules coopèrent pour assurer une surveillance intelligente et continue. Il met en évidence le rôle central de la Raspberry Pi comme unité de collecte et de traitement initial, la coordination assurée par le backend, ainsi que la diffusion des résultats vers le frontend destiné aux parents.

Au-delà de la simple répartition des composants, l'architecture montre la logique d'intégration et de circulation des informations : les capteurs et la caméra fournissent les données brutes, le backend les fusionne et les analyse grâce au moteur d'intelligence artificielle, puis le frontend les restitue sous une forme claire et exploitable. Cette organisation hiérarchisée garantit la cohérence du système et la fiabilité des alertes générées.

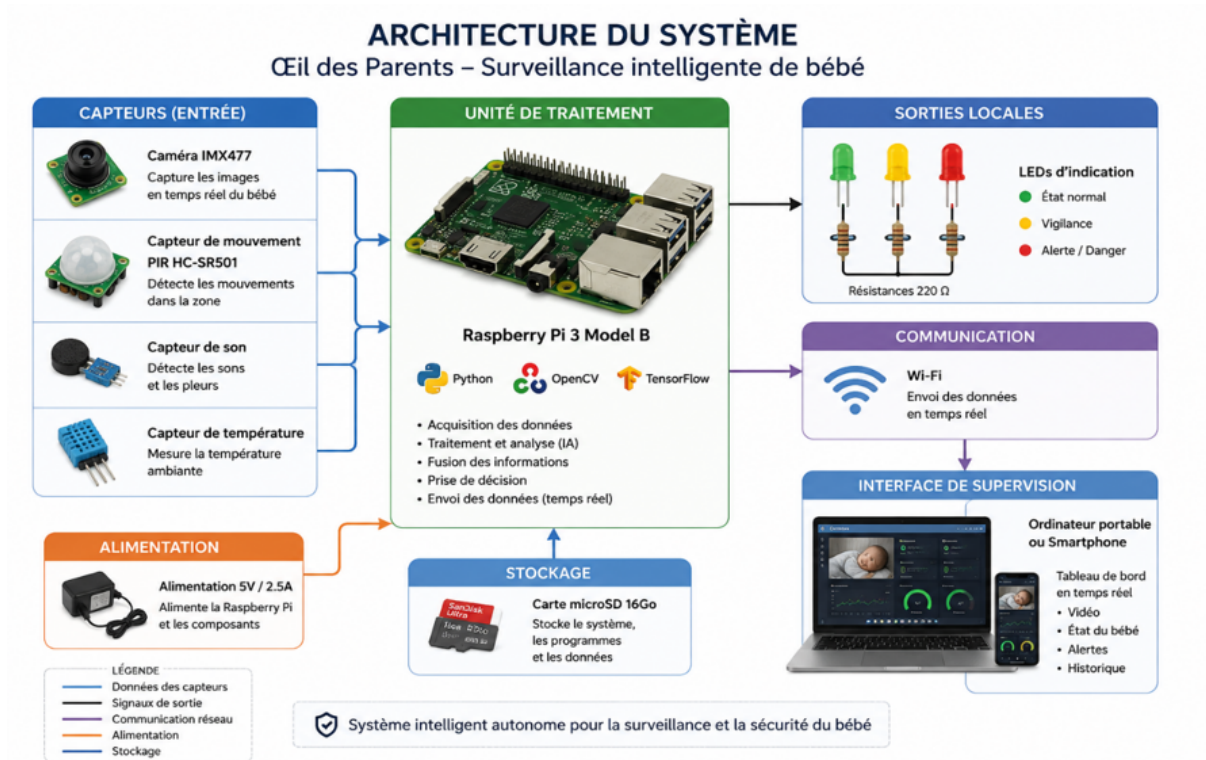


FIGURE 3.1 – Architecture du système

3.2.2 Montage du dispositif

Le montage du dispositif représente une étape cruciale dans la concrétisation du projet, car il assure la mise en relation entre les différents composants matériels et leur intégration dans une architecture cohérente. La Raspberry Pi, véritable cœur du système, est reliée aux capteurs de mouvement, de son et de température, ainsi qu'à la caméra, afin de collecter en continu les données nécessaires à la surveillance. Les LED et les résistances complètent l'installation en fournissant des indicateurs visuels sur l'état du système.

Ce processus de montage ne se limite pas à une simple connexion physique : il implique également une configuration minutieuse des interfaces matérielles et logicielles, garantissant que chaque capteur communique correctement avec l'unité centrale. Ainsi, le dispositif devient une plateforme fonctionnelle capable de transformer les signaux bruts en informations exploitables, prêtes à être transmises au backend pour analyse. Cette étape illustre la transition entre la conception théorique et la mise en œuvre pratique, et constitue le socle technique sur lequel repose l'ensemble du système de surveillance intelligente.

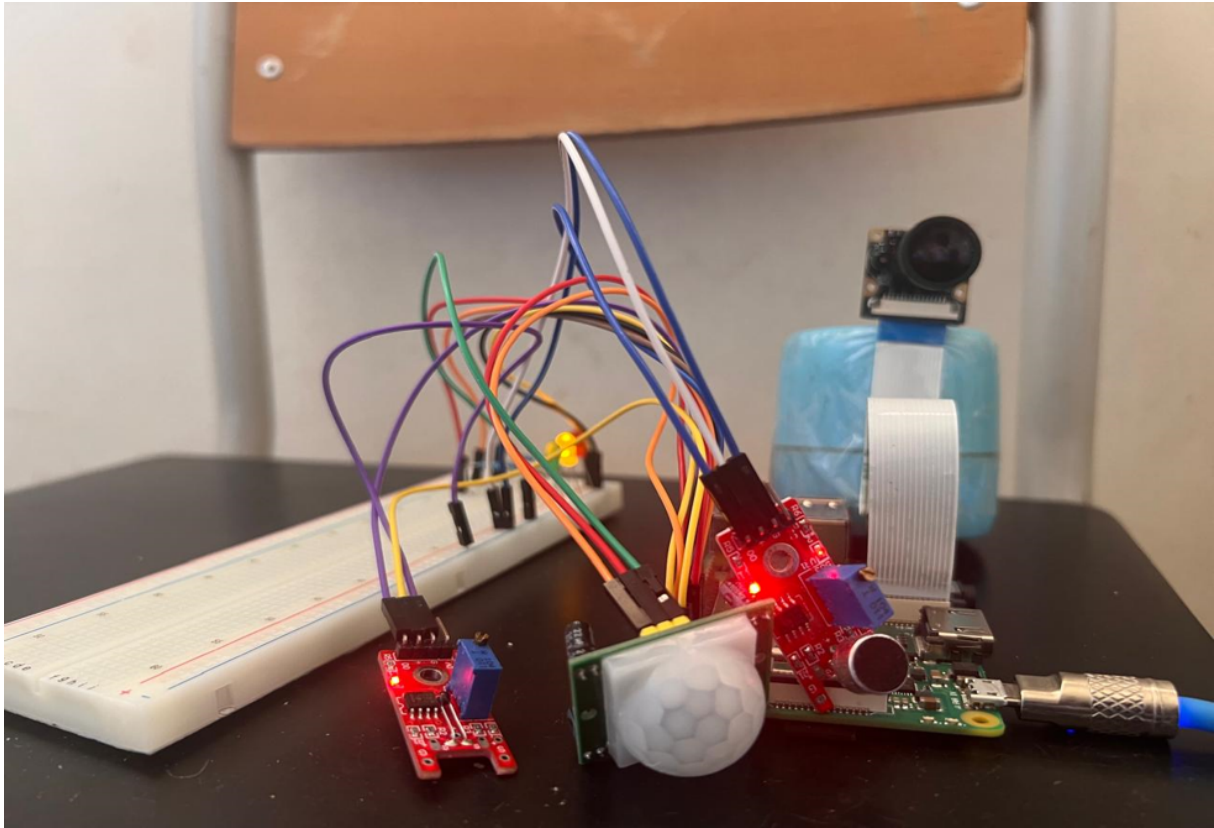


FIGURE 3.2 – Montage des matériels

3.3 Raspberry Pi

La Raspberry Pi qui est l'unité centrale de notre système, collecte les données des capteurs, exécute les algorithmes d'analyse (vision et audio), pilote les périphériques et transmet les informations vers le backend du système. En fin que la transmission des données des capteurs et camera soient possible, nous sommes passés par les étapes ci-après :

3.3.1 Installation du système d'exploitation

La première étape a consisté à installer Raspberry Pi OS sur une carte microSD de 16 Go à l'aide de Raspberry Pi Imager. Ce système d'exploitation permet l'exécution des programmes développés pour le projet.

3.3.2 Configuration de la Raspberry Pi

Après le premier démarrage, la Raspberry Pi a été configurée en activant les interfaces matérielles nécessaires, notamment :

- L'interface camera ;
- Les broches GPIO ;
- La connexion réseau (Wi-Fi).

Le système a ensuite été mis à jour afin d'assurer la compatibilité des bibliothèques utilisées.

3.3.3 Installation des bibliothèques

Les bibliothèques nécessaires au fonctionnement des capteurs ont ensuite été installées, notamment :

- Python;
- OpenCV (traitement d'images);
- Picamera2 (gestion de la caméra Raspberry Pi);
- Les bibliothèques GPIO permettant la communication avec les différents capteurs.

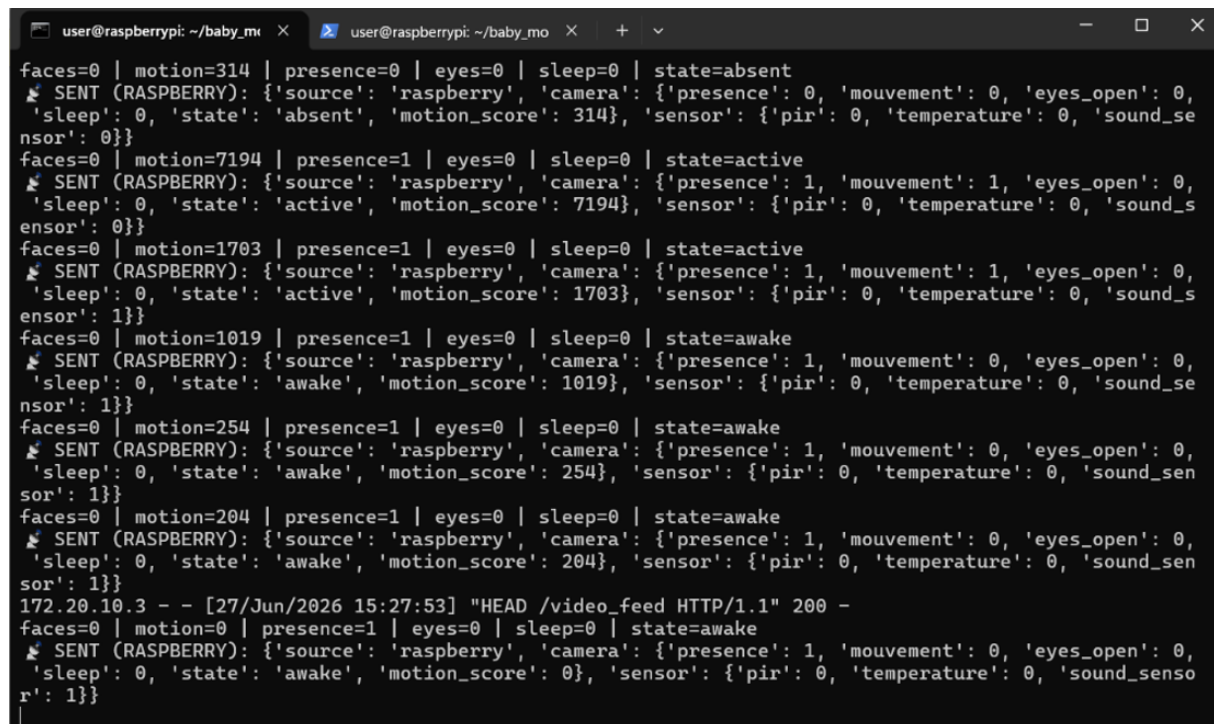
3.3.4 Acquisition des données

Des programmes Python ont été développés afin de :

- Capturer les images de la caméra en temps réel;
- Détecter la présence et les mouvements;
- Déterminer l'état des yeux (ouverts ou fermés);
- Lire les informations provenant des capteurs PIR, sonore et de température.

Toutes ces informations sont regroupées dans une structure de données unique.

3.3.5 IMAGE de l'exécution du programme main.py de la Raspberry Pi dans le terminal



```

user@raspberrypi: ~/baby_mo x user@raspberrypi: ~/baby_mo x + v
faces=0 | motion=314 | presence=0 | eyes=0 | sleep=0 | state=absent
SENT (RASPBerry): {'source': 'raspberry', 'camera': {'presence': 0, 'mouvement': 0, 'eyes_open': 0, 'sleep': 0, 'state': 'absent', 'motion_score': 314}, 'sensor': {'pir': 0, 'temperature': 0, 'sound_sensor': 0}}
faces=0 | motion=7194 | presence=1 | eyes=0 | sleep=0 | state=active
SENT (RASPBerry): {'source': 'raspberry', 'camera': {'presence': 1, 'mouvement': 1, 'eyes_open': 0, 'sleep': 0, 'state': 'active', 'motion_score': 7194}, 'sensor': {'pir': 0, 'temperature': 0, 'sound_sensor': 0}}
faces=0 | motion=1703 | presence=1 | eyes=0 | sleep=0 | state=active
SENT (RASPBerry): {'source': 'raspberry', 'camera': {'presence': 1, 'mouvement': 1, 'eyes_open': 0, 'sleep': 0, 'state': 'active', 'motion_score': 1703}, 'sensor': {'pir': 0, 'temperature': 0, 'sound_sensor': 1}}
faces=0 | motion=1019 | presence=1 | eyes=0 | sleep=0 | state=awake
SENT (RASPBerry): {'source': 'raspberry', 'camera': {'presence': 1, 'mouvement': 0, 'eyes_open': 0, 'sleep': 0, 'state': 'awake', 'motion_score': 1019}, 'sensor': {'pir': 0, 'temperature': 0, 'sound_sensor': 1}}
faces=0 | motion=254 | presence=1 | eyes=0 | sleep=0 | state=awake
SENT (RASPBerry): {'source': 'raspberry', 'camera': {'presence': 1, 'mouvement': 0, 'eyes_open': 0, 'sleep': 0, 'state': 'awake', 'motion_score': 254}, 'sensor': {'pir': 0, 'temperature': 0, 'sound_sensor': 1}}
faces=0 | motion=204 | presence=1 | eyes=0 | sleep=0 | state=awake
SENT (RASPBerry): {'source': 'raspberry', 'camera': {'presence': 1, 'mouvement': 0, 'eyes_open': 0, 'sleep': 0, 'state': 'awake', 'motion_score': 204}, 'sensor': {'pir': 0, 'temperature': 0, 'sound_sensor': 1}}
172.20.10.3 - - [27/Jun/2026 15:27:53] "HEAD /video_feed HTTP/1.1" 200 -
faces=0 | motion=0 | presence=1 | eyes=0 | sleep=0 | state=awake
SENT (RASPBerry): {'source': 'raspberry', 'camera': {'presence': 1, 'mouvement': 0, 'eyes_open': 0, 'sleep': 0, 'state': 'awake', 'motion_score': 0}, 'sensor': {'pir': 0, 'temperature': 0, 'sound_sensor': 1}}

```

FIGURE 3.3 – Image d'exécution

3.3.6 Transmission des données vers le backend

Une fois les données collectées, elles sont transmises au backend via une connexion wifi qu'on a définie. Le backend reçoit les informations issues de la Raspberry Pi, les fusionne

avec les autres données disponibles, puis les transmet au moteur d’intelligence artificielle chargé de déterminer l’état du bébé et de calculer le score de danger.

3.4 Backend

Le backend constitue le cœur logiciel du système. Il assure la réception des données provenant de la Raspberry Pi et du module d’acquisition audio, coordonne les différents services d’analyse, puis transmet les résultats en temps réel à l’interface web destinée aux parents. Le backend a été développé en Python et organisé de manière modulaire afin de faciliter son évolution et sa maintenance.

3.4.1 Organisation du backend



FIGURE 3.4 – La structure du backend

Fichiers	Rôles
App.py	Point d’entrée de l’application. Initialise le serveur, reçoit les données et assure leur diffusion vers le frontend.
pc_audio_client.py	Capture le flux audio du microphone, réalise le prétraitement et transmet les résultats au backend.
audio_model.pkl	Modèle d’IA entraîné pour la classification des sons (pleurs, cris, voix).
labels.pkl	Contient les classes utilisées par le modèle de reconnaissance audio.
ai_engine.py	Moteur d’IA chargé de fusionner les données, calculer le score de danger et générer les alertes.

TABLE 3.2 – Organisation des fichiers du backend

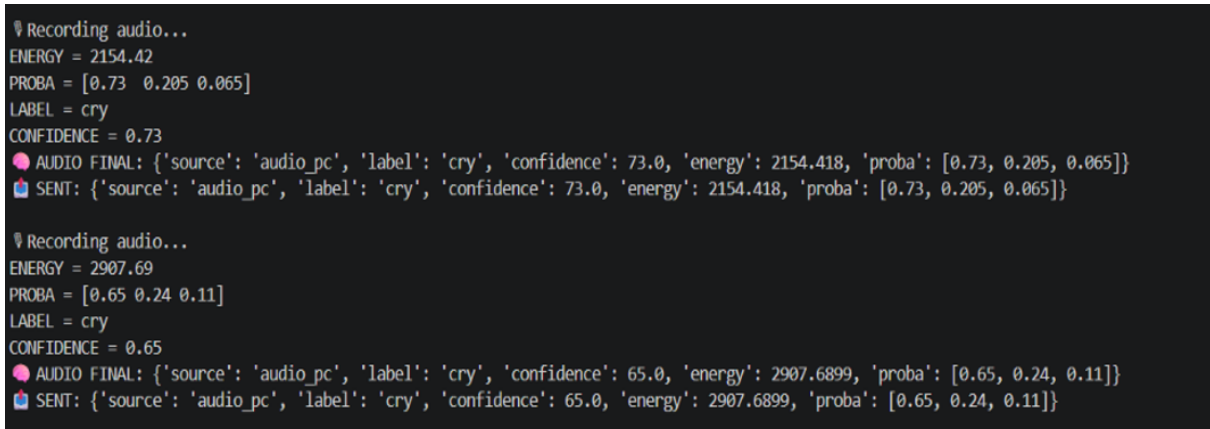
3.4.2 Ai engine

Le fichier **aiengine**, constitue le moteur décisionnel du système. Développé en Python, il reçoit les données issues de la caméra, de l’analyse audio et des différents capteurs, puis réalise une fusion de ces informations afin d’évaluer l’état global du bébé. Il assure notamment l’analyse des événements détectés, le calcul du score de danger, la détermination de

l'activité du bébé (éveillé, endormi, en pleurs ou absent), la stabilisation des changements d'état afin de réduire les faux positifs, la génération des alertes ainsi que la prédiction du comportement à court terme. Pour améliorer la fiabilité des décisions, le moteur conserve également un historique des dernières observations grâce à la structure de données **deque** de la bibliothèque **collections**, tandis que le module **time** est utilisé pour horodater les événements. Les résultats produits sont ensuite transmis au backend, qui les diffuse en temps réel vers l'interface de supervision des parents.

3.4.3 Pc audio client

Le fichier **pcaudioclient** constitue le module d'acquisition et d'analyse audio du système. Développé en Python, il assure l'enregistrement en continu des sons à l'aide du microphone du pc, puis réalise leur prétraitement avant de les soumettre au modèle d'intelligence artificielle. Les caractéristiques acoustiques sont extraites grâce à la bibliothèque **Librosa** (coefficients MFCC), puis analysées par un modèle de classification chargé avec **Joblib** (audiomodel). Afin d'améliorer la fiabilité des résultats, le module effectue une normalisation du signal, calcule son niveau d'énergie et applique un seuil de confiance permettant de distinguer les pleurs, les bruits, le silence ou la voix. Enfin, les informations produites (classe détectée, niveau de confiance, énergie du signal et probabilités de classification) sont transmises en temps réel au backend via des requêtes HTTP grâce à la bibliothèque **Requests**, où elles seront intégrées au processus décisionnel du système.



```
❯ Recording audio...
ENERGY = 2154.42
PROBA = [0.73 0.205 0.065]
LABEL = cry
CONFIDENCE = 0.73
🔴 AUDIO FINAL: {'source': 'audio_pc', 'label': 'cry', 'confidence': 73.0, 'energy': 2154.418, 'proba': [0.73, 0.205, 0.065]}
📡 SENT: {'source': 'audio_pc', 'label': 'cry', 'confidence': 73.0, 'energy': 2154.418, 'proba': [0.73, 0.205, 0.065]}

❯ Recording audio...
ENERGY = 2907.69
PROBA = [0.65 0.24 0.11]
LABEL = cry
CONFIDENCE = 0.65
🔴 AUDIO FINAL: {'source': 'audio_pc', 'label': 'cry', 'confidence': 65.0, 'energy': 2907.6899, 'proba': [0.65, 0.24, 0.11]}
📡 SENT: {'source': 'audio_pc', 'label': 'cry', 'confidence': 65.0, 'energy': 2907.6899, 'proba': [0.65, 0.24, 0.11]}
```

FIGURE 3.5 – image d'exécution de *pc_{audio}client*

3.4.4 App

Le fichier **app.py** constitue le point d'entrée du backend de l'application. Développé en Python à l'aide des Framework Flask et Flask-SocketIO, il assure la communication entre les différents modules du système. Son rôle principal est de recevoir les données transmises par la Raspberry Pi (caméra et capteurs) ainsi que par le module d'analyse audio, de centraliser ces informations dans un état global, puis de les transmettre au moteur décisionnel AI Engine pour analyse. Après traitement, les résultats produits (activité du bébé, score de danger, état du système, alertes et prédictions) sont diffusés en temps réel vers l'interface web grâce aux WebSockets, permettant une mise à jour instantanée du tableau de bord des parents. Le serveur gère également les requêtes HTTP, le partage de

ressources via CORS, ainsi qu'un mécanisme de temporisation garantissant la cohérence des données audio en cas d'absence temporaire de communication.

```
presence=1 | eyes=0 | sleep=0 | audio=cry | activity=crying | candidate=crying | pending=None | count=0 | score=90
DATA IA: {'activite': 'crying', 'prediction': 'baby_may_wake_up', 'etat_systeme': 'ALERTE', 'score_danger': 90, 'alertes': ['baby_cry'], 'confidence_audio': 64.5, 'data': {'camera': {'presence': 1, 'mouvement': 0, 'eyes_open': 0, 'sleep': 0, 'state': 'awake', 'motion_score': 509}, 'audio': {'label': 'cry', 'confidence': 64.5, 'energy': 2066.2285, 'proba': [0.645, 0.295, 0.06]}, 'sensor': {'pir': 0, 'temperature': 0, 'sound_sensor': 1}}}}
172.20.10.2 - - [27/Jun/2026 16:45:34] "POST /data HTTP/1.1" 200 765 0.001607
(32744) accepted ('172.20.10.2', 60594)
presence=1 | eyes=0 | sleep=0 | audio=cry | activity=crying | candidate=crying | pending=None | count=0 | score=90
DATA IA: {'activite': 'crying', 'prediction': 'baby_may_wake_up', 'etat_systeme': 'ALERTE', 'score_danger': 90, 'alertes': ['baby_cry'], 'confidence_audio': 64.5, 'data': {'camera': {'presence': 1, 'mouvement': 0, 'eyes_open': 0, 'sleep': 0, 'state': 'awake', 'motion_score': 959}, 'audio': {'label': 'cry', 'confidence': 64.5, 'energy': 2066.2285, 'proba': [0.645, 0.295, 0.06]}, 'sensor': {'pir': 0, 'temperature': 0, 'sound_sensor': 1}}}}
172.20.10.2 - - [27/Jun/2026 16:45:35] "POST /data HTTP/1.1" 200 765 0.001286
(32744) accepted ('172.20.10.3', 58382)
AUDIO REQU: {'label': 'cry', 'confidence': 64.0, 'energy': 1296.0323, 'proba': [0.64, 0.265, 0.095]}
presence=1 | eyes=0 | sleep=0 | audio=cry | activity=crying | candidate=crying | pending=None | count=0 | score=90
DATA IA: {'activite': 'crying', 'prediction': 'baby_may_wake_up', 'etat_systeme': 'ALERTE', 'score_danger': 90, 'alertes': ['baby_cry'], 'confidence_audio': 64.0, 'data': {'camera': {'presence': 1, 'mouvement': 0, 'eyes_open': 0, 'sleep': 0, 'state': 'awake', 'motion_score': 959}, 'audio': {'label': 'cry', 'confidence': 64.0, 'energy': 1296.0323, 'proba': [0.64, 0.265, 0.095]}, 'sensor': {'pir': 0, 'temperature': 0, 'sound_sensor': 1}}}}
172.20.10.3 - - [27/Jun/2026 16:45:36] "POST /data HTTP/1.1" 200 765 0.000901
(32744) accepted ('172.20.10.2', 39678)
presence=1 | eyes=0 | sleep=0 | audio=cry | activity=crying | candidate=crying | pending=None | count=0 | score=90
DATA IA: {'activite': 'crying', 'prediction': 'baby_may_wake_up', 'etat_systeme': 'ALERTE', 'score_danger': 90, 'alertes': ['baby_cry'], 'confidence_audio': 64.0, 'data': {'camera': {'presence': 1, 'mouvement': 0, 'eyes_open': 0, 'sleep': 0, 'state': 'awake', 'motion_score': 278}, 'audio': {'label': 'cry', 'confidence': 64.0, 'energy': 1296.0323, 'proba': [0.64, 0.265, 0.095]}, 'sensor': {'pir': 0, 'temperature': 0, 'sound_sensor': 1}}}}
172.20.10.2 - - [27/Jun/2026 16:45:37] "POST /data HTTP/1.1" 200 765 0.001081
(32744) accepted ('172.20.10.2', 39680)
```

FIGURE 3.6 – image d'exécution de app.py

3.4.5 Choix de la technologie

3.4.5.1 Visual Studio Code

Visual studio code est un éditeur de code extensible développé par Microsoft pour Windows, Linux et MacOS. Il offre des fonctionnalités avancées telles que le débogage, la mise en évidence de la syntaxe, la complétion intelligente du code et l'intégration de Git. Sa flexibilité et ses nombreuses extensions en font un outil précieux pour améliorer la productivité et la qualité du code lors du développement. Dans notre projet, nous avons utilisé Visual Studio Code pour l'édition et la gestion du code source.

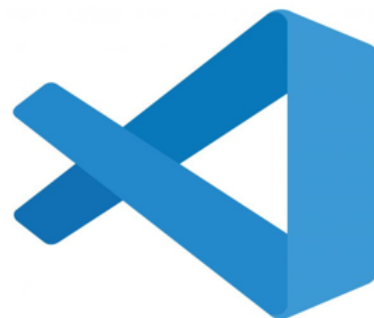


FIGURE 3.7 – visual studio code

3.4.5.2 Python

Nous avons choisi Python pour le développement du backend, car il offre un environnement optimal pour la gestion des données et l'intégration des capteurs. Grâce à ses nombreuses bibliothèques, il nous a permis de collecter, traiter et analyser les informations issues du Raspberry Pi, tout en assurant la communication avec l'interface frontend.



FIGURE 3.8 – Python

3.5 Frontend

Le frontend représente la couche de présentation du système et constitue l'interface principale avec laquelle les parents interagissent. Développé avec **React.js**, il offre une visualisation en temps réel des informations transmises par le backend, sous une forme claire, intuitive et ergonomique. L'interface est structurée en plusieurs pages et composants réutilisables, tels que le tableau de bord, la page de suivi de l'état du bébé, les alertes, les capteurs et les paramètres. Grâce à **React Router**, la navigation entre ces différentes sections est fluide et rapide, tandis que **Socket.IO Client** assure une communication bidirectionnelle avec le backend, permettant de recevoir instantanément les mises à jour sans nécessiter de rechargement de la page.

Les données reçues sont présentées sous forme de cartes d'information, de graphiques dynamiques, d'indicateurs de danger, de recommandations et d'historiques d'événements. Cette organisation visuelle facilite la compréhension et permet aux parents de suivre l'évolution de la situation de manière intuitive. La mise en page, réalisée avec CSS3, garantit une interface responsive, moderne et adaptée à différents supports (ordinateur, tablette, smartphone), ce qui renforce l'accessibilité et l'efficacité du système.

Au-delà de la simple présentation des données, le frontend joue un rôle essentiel dans l'expérience utilisateur. Il traduit les résultats complexes issus du moteur d'intelligence artificielle en informations simples et exploitables, tout en offrant des fonctionnalités interactives comme la consultation des flux vidéo, l'examen des alertes en temps réel, ou encore la personnalisation des paramètres de surveillance. Cette couche de présentation constitue donc le lien direct entre la technologie et l'utilisateur final, en rendant la supervision du bébé non seulement possible mais aussi pratique et agréable.

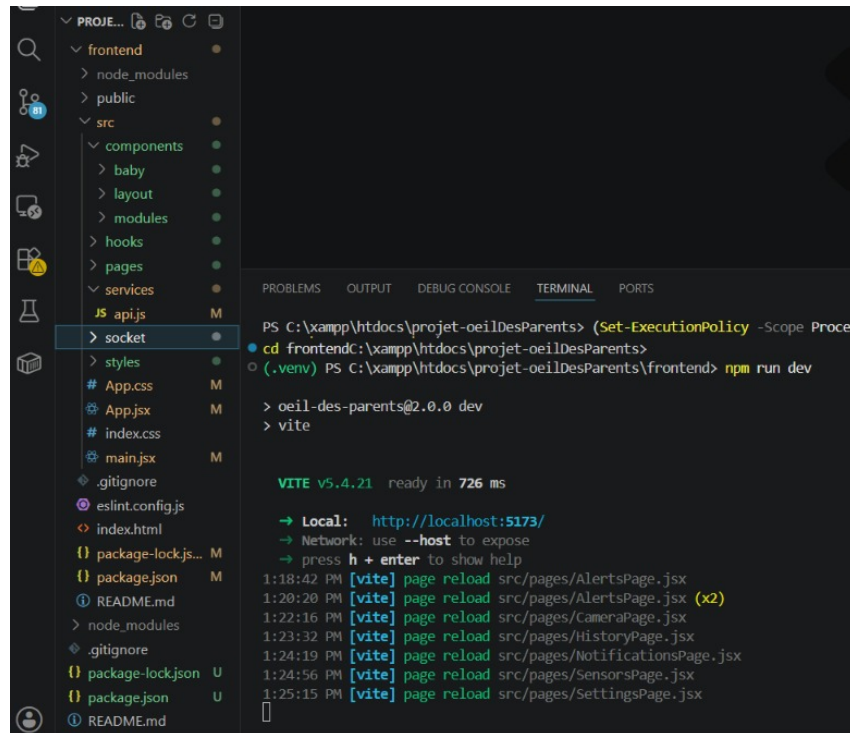


FIGURE 3.9 – La structure du frontend

Fichiers/Dossiers	Rôles
components/	Composants React réutilisables (dashboard, état du bébé, alertes, graphiques, etc.).
hooks/	Hooks personnalisés pour la gestion des données reçues en temps réel via WebSocket.
pages/	Pages de l'application (Dashboard, État du bébé, Alertes, Capteurs, Paramètres).
services/	Services de communication avec le backend et gestion des traitements spécifiques.
socket/	Connexion Socket.IO entre frontend et backend pour mises à jour en temps réel.
styles/	Feuilles de style CSS pour la mise en forme de l'interface utilisateur.
App.jsx	Composant principal, responsable du routage et de l'organisation générale.
App.css	Feuille de style principale associée au composant App.
main.jsx	Point d'entrée de l'application React, initialise et monte l'application dans le navigateur.
index.css	Feuille de style globale appliquée à l'ensemble de l'application.

TABLE 3.3 – Organisation des fichiers du frontend

3.5.1 INTERFACE DU DASHBOARD

Le tableau de bord constitue l'interface principale de supervision du système. Il regroupe en temps réel les informations essentielles relatives à l'état du bébé et au fonctionnement du système. Les principaux modules qui composent le tableau de bord sont les suivants :

- **État du bébé (Baby Status)** : ce module présente l'activité actuelle du bébé (éveillé, endormi, en pleurs ou absent) ainsi que son état général
- **Vue de la caméra (Camera View)** : il affiche le flux vidéo provenant de la caméra installée près du bébé afin de permettre une surveillance visuelle en temps réel.
- **Score de danger (Danger Score)** : ce module indique le niveau de risque calculé par le moteur décisionnel sous forme d'un score compris entre 0 et 100.
- **Prédiction IA (AI Prediction)** : il affiche la prédiction générée par le moteur d'intelligence artificielle concernant l'évolution probable de l'état du bébé.
- **Analyse audio (Audio Panel)** : ce module présente le résultat de l'analyse sonore, notamment le type de son détecté (silence, pleurs ou bruit) ainsi que le niveau de confiance associé.
- **État du système (System Health)** : il informe l'utilisateur sur le bon fonctionnement des différents composants matériels et logiciels du système.
- **Alertes (Alerts)** : ce module regroupe les alertes générées par le moteur décisionnel lorsqu'une situation inhabituelle ou dangereuse est détectée.
- **Évolution du score de danger (Danger Graph)** : il représente graphiquement l'évolution du score de danger au cours du temps afin de faciliter le suivi de la situation. L'ensemble de ces modules est mis à jour automatiquement grâce à une communication en temps réel entre le backend et le frontend, permettant aux parents de suivre en permanence l'état de leur bébé.

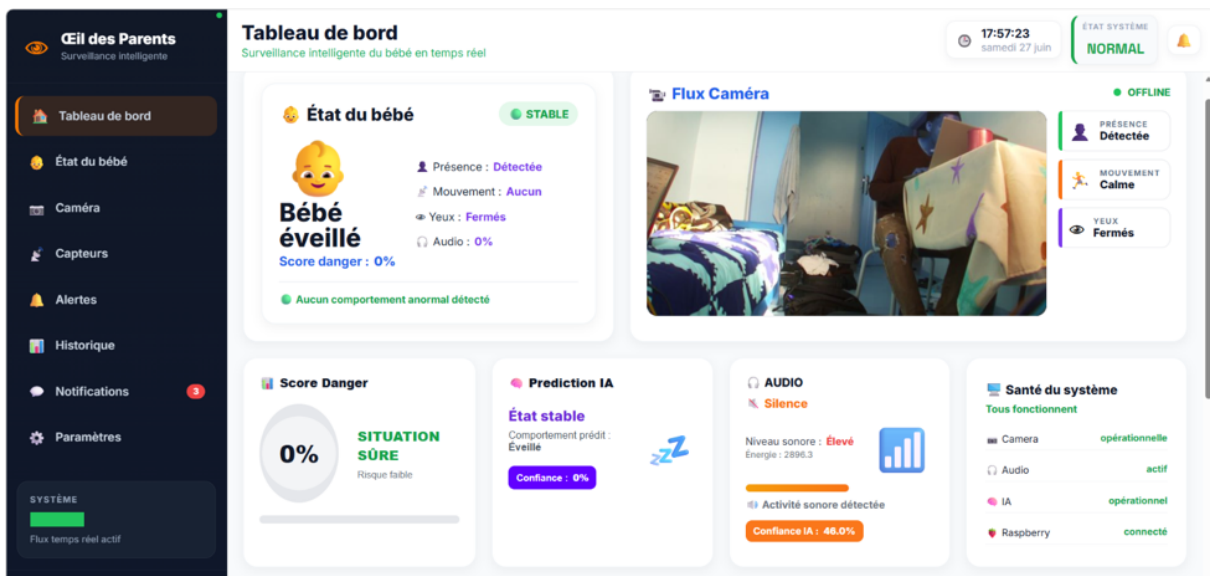


FIGURE 3.10 – dasbord

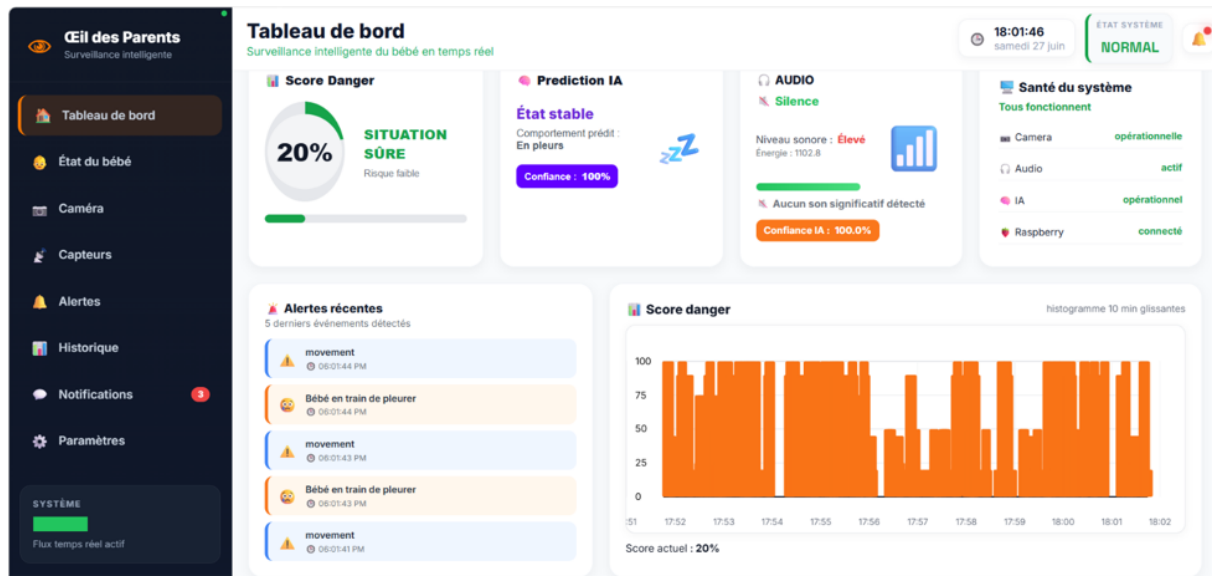


FIGURE 3.11 – dasbord

3.5.2 ETAT DU BEBE

La page État du bébé fournit une analyse plus détaillée de la situation du bébé. Elle rassemble les informations essentielles produites par le système afin de permettre aux parents de suivre son état de manière claire et complète.

Les principaux modules qui composent cette page sont les suivants :

- **Vue d'ensemble (Baby Overview)** : ce module présente les informations générales sur l'état actuel du bébé, notamment son activité et son statut en temps réel.
- **Résumé instantané (Baby Summary)** : il affiche les principaux indicateurs du système, tels que le score de danger, l'état général et les informations importantes à consulter rapidement.
- **Analyse comportementale (Baby Behavior)** : ce module regroupe les informations issues de la caméra et des capteurs afin de décrire le comportement observé du bébé (présence, mouvements, état des yeux, sommeil, etc.).
- **Évolution du danger (Danger Evolution)** : il représente graphiquement l'évolution du score de danger au fil du temps afin de visualiser rapidement les variations du niveau de risque.
- **Recommandations de l'IA (AI Recommendation)** : ce module fournit des conseils et recommandations générés automatiquement par le moteur décisionnel en fonction de la situation détectée.
- **Historique des événements (History)** : il conserve les derniers événements enregistrés par le système, permettant aux parents de consulter les activités et alertes récentes. L'ensemble de ces modules est actualisé automatiquement grâce aux données reçues en temps réel depuis le backend, offrant ainsi une vision détaillée et continue de l'état du bébé.

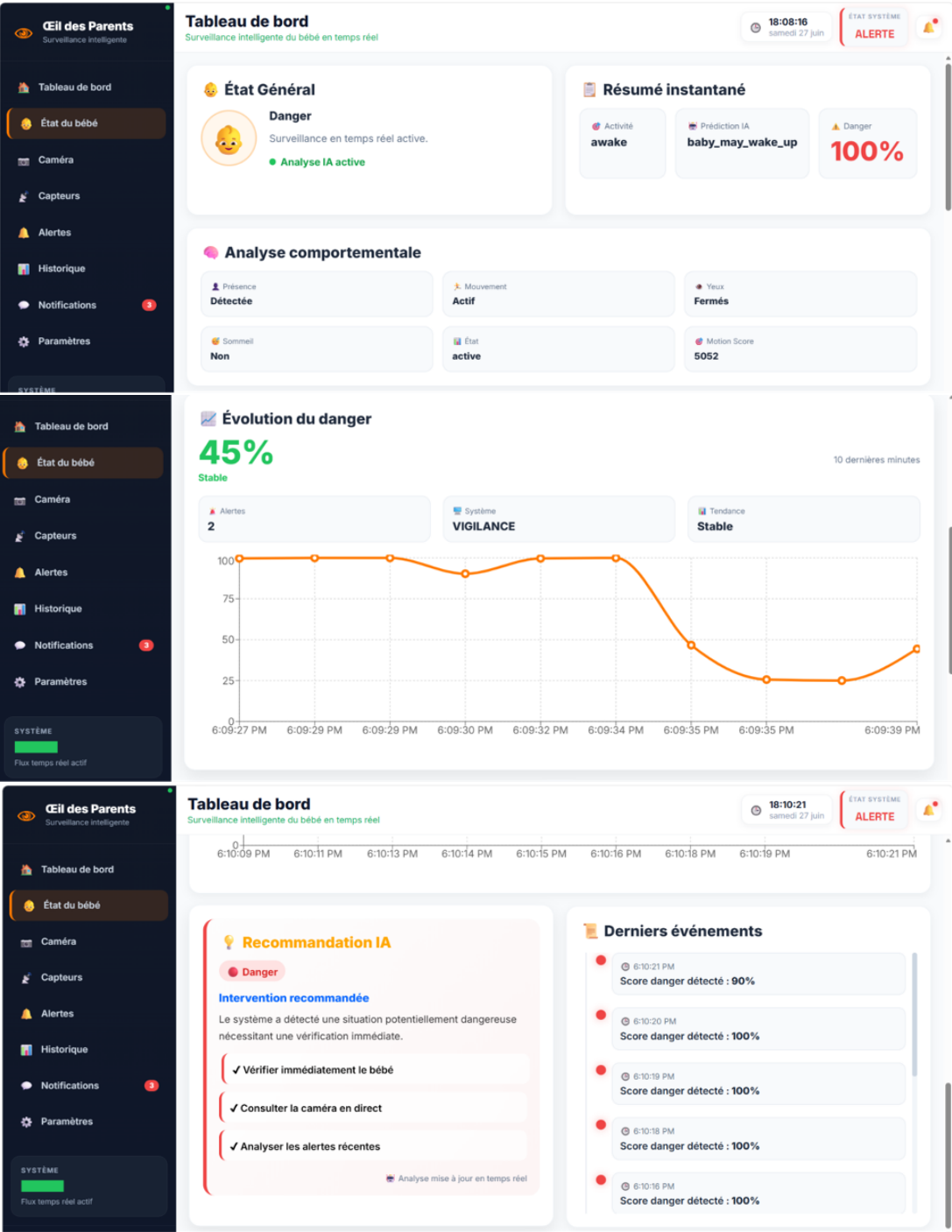


FIGURE 3.12 – Tableaux de bord 3, 4 et 5

3.5.3 AUTRES PAGES DISPONIBLES DANS LA BARRE LATÉRALE

- **Alertes** : regroupe l'ensemble des alertes générées par le système afin d'informer rapidement les parents des situations nécessitant une attention particulière.
- **Capteurs** : présente les informations provenant des différents capteurs (PIR, température, son, etc.) ainsi que leur état de fonctionnement.
- **Paramètres** : permet de configurer les différents paramètres de l'application, tels que les préférences utilisateur, les seuils d'alerte et les options de surveillance

Conclusion Générale

Au terme de ce projet de fin d'études, nous avons conçu et réalisé Œil des Parents, un système intelligent de surveillance de bébé combinant les technologies de l'Internet des Objets (IoT), de la vision par ordinateur, de l'analyse audio et du développement web. L'objectif principal était de proposer une solution capable d'assister les parents dans la surveillance quotidienne de leur enfant tout en offrant un suivi en temps réel de son état.

Le système développé permet de collecter les informations provenant de la caméra et des différents capteurs installés autour du bébé, puis de les transmettre au backend où elles sont analysées par un moteur décisionnel. À partir de ces données, Œil des Parents est capable d'identifier l'activité du bébé, de calculer un score de danger, de générer des alertes, de proposer des recommandations adaptées et d'afficher l'ensemble des résultats sur une interface web moderne et interactive.

La réalisation de ce projet nous a permis de mettre en pratique les connaissances acquises au cours de notre formation dans plusieurs domaines complémentaires, notamment les systèmes embarqués, l'Internet des Objets, le développement d'applications web, les communications en temps réel ainsi que l'intelligence artificielle. Elle nous a également permis de développer des compétences en conception logicielle, en intégration de composants matériels et logiciels, ainsi qu'en gestion d'un projet complet.

Les résultats obtenus montrent que Œil des Parents constitue une solution fonctionnelle capable d'assurer une surveillance intelligente du bébé tout en facilitant la prise de décision des parents grâce aux informations présentées de manière claire et instantanée. Bien que le système réponde aux principaux objectifs fixés, plusieurs pistes d'amélioration peuvent être envisagées, telles que l'utilisation de modèles d'intelligence artificielle plus performants, l'ajout d'une application mobile, la reconnaissance de comportements plus complexes, l'intégration de nouveaux capteurs biométriques ou encore le déploiement sur une infrastructure cloud afin d'améliorer les performances et l'accessibilité.

En définitive, Œil des Parents représente une base solide pour le développement d'une solution de surveillance intelligente plus complète. Ce projet démontre l'intérêt de combiner les technologies de l'IoT et de l'intelligence artificielle afin de concevoir des systèmes innovants capables d'améliorer le confort, la sécurité et la tranquillité d'esprit des parents dans la surveillance de leur enfant.

Bibliographie

- [1] Raspberry Pi Foundation, Raspberry Pi Documentation.
Disponible sur : <https://www.raspberrypi.com/documentation/>
- [2] OpenCV, OpenCV Documentation.
Disponible sur : <https://docs.opencv.org/>
- [3] Flask Documentation, Flask Web Development Framework.
Disponible sur : <https://flask.palletsprojects.com/>
- [4] Flask-SocketIO Documentation.
Disponible sur : <https://flask-socketio.readthedocs.io/>
- [5] Socket.IO Documentation.
Disponible sur : <https://socket.io/docs/>
- [6] React Documentation.
Disponible sur : <https://react.dev/>
- [7] Librosa Documentation.
Disponible sur : <https://librosa.org/doc/>
- [8] TensorFlow Documentation.
Disponible sur : <https://www.tensorflow.org/>
- [9] NumPy Documentation.
Disponible sur : <https://numpy.org/doc/>
- [10] Scikit-learn Documentation.
Disponible sur : <https://scikit-learn.org/>
- [11] Picamera2 Documentation.
Disponible sur : <https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf>
- [12] Python Software Foundation, Python Documentation.
Disponible sur : <https://docs.python.org/>
- [13] Mozilla Developer Network (MDN), HTML, CSS et JavaScript Documentation.
Disponible sur : <https://developer.mozilla.org/>
- [14] W3Schools, HTML, CSS, JavaScript and React Tutorials.
Disponible sur : <https://www.w3schools.com/>

Annexe

Configuration matérielle

Matériel utilisé

- Raspberry Pi 3 Model B
- Caméra Raspberry Pi IMX477
- Capteur de mouvement PIR HC-SR501
- Capteur de son
- Capteur de température
- Trois LED (verte, jaune et rouge)
- Résistances de $220\ \Omega$
- Breadboard
- Carte microSD 16 Go
- Ordinateur portable

Environnement logiciel

Système d'exploitation

- Raspberry Pi OS

Langages de programmation

- Python
- JavaScript
- HTML5
- CSS3

Frameworks et bibliothèques

- React.js
- Flask
- Flask-SocketIO
- Socket.IO
- OpenCV
- Picamera2
- NumPy
- Librosa
- Scikit-learn

Architecture du système

Le système **Œil des Parents** est composé de quatre principaux modules :

- Module d'acquisition des données (caméra et capteurs)
- Backend de traitement et moteur décisionnel (AI Engine)
- Base de traitement audio
- Interface Web de supervision en temps réel

Structure du projet

Backend

- app.py
- analyse_service.py
- services/
 - ai_engine.py

Frontend

- components/
- pages/
- hooks/
- services/
- socket/
- styles/

Interfaces développées

Les principales interfaces réalisées sont :

- Tableau de bord (Dashboard)
- Page « État du bébé »
- Page des alertes
- Page des capteurs
- Page des paramètres

Fonctionnalités réalisées

Le système **Œil des Parents** permet notamment :

- Détection de la présence du bébé.
- Détection des mouvements.
- Détection de l'ouverture et de la fermeture des yeux.
- Détection de l'état de sommeil.
- Analyse des sons (pleurs, bruit, silence).
- Mesure de la température ambiante.
- Détection des mouvements grâce au capteur PIR.
- Calcul d'un score de danger en temps réel.
- Génération d'alertes intelligentes.
- Prédiction de l'activité du bébé.

- Génération de recommandations basées sur l'intelligence artificielle.
- Visualisation des données sous forme de tableaux de bord, de graphiques et d'indicateurs en temps réel.

Perspectives d'amélioration

Les améliorations envisageables comprennent :

- Développement d'une application mobile Android et iOS.
- Utilisation de modèles d'intelligence artificielle plus performants.
- Ajout de nouveaux capteurs biométriques.
- Déploiement sur une infrastructure Cloud.
- Enregistrement et consultation de l'historique dans une base de données.
- Notifications par SMS, courrier électronique ou notification push.
- Reconnaissance d'un plus grand nombre de comportements et d'activités du bébé.
- Authentification sécurisée des utilisateurs.
- Gestion de plusieurs caméras et de plusieurs chambres.

